

C Programming

Array: Ch.10

Declaration & initialization | 1D & multidimensional arrays

What You Will Learn in This Chapter

- The concept of arrays
- Declaration and initialization of arrays
- One-dimensional **array**
- Multidimensional **array**
- Arrays allow you to allocate space to store multiple values at once.

Array

- **Arrays** allow you to create multiple variables at once: `int s[10];`
- The need for **arrays**: with separate variable names, it's hard to manipulate related data, since you would have to remember each name.
- **Array** characteristic: arrays are stored in a contiguous space in memory.
For example, `s[0]` and `s[1]` are physically adjacent to each other in memory.
- The biggest advantage of an **array** is that you can access and process related data sequentially, using one shared name and different **index** numbers.

Array

What is an Array in C?

An array is a collection of elements of the same type stored in a **contiguous memory** location.

Variable



One value, one name

Array



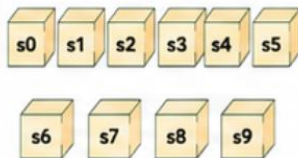
Many values, one name

1) Using Regular Variables

// each variable has a separate name

```
int s0;  
int s1;  
int s2;  
...  
int s9;
```

✗ **Hard to manipulate!**
Many names, scattered memory

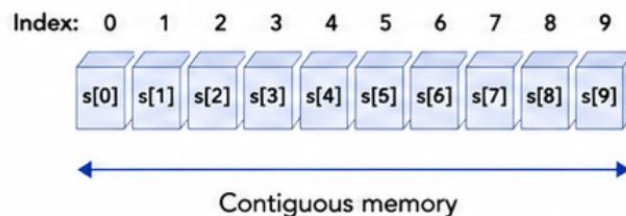


2) Using Array

// one name for many elements

```
int s[10];
```

✓ **Easy to access and process!**
Stored in a contiguous space



3) Memory Representation of an Array

```
int arr[10];
```

Address (Hex)	Array Element	Value
0x0000	arr[0]	10
0x0004	arr[1]	20
0x0008	arr[2]	30
0x000C	arr[3]	40
0x0010	arr[4]	50
0x0014	arr[5]	60
0x0018	arr[6]	70
0x001C	arr[7]	80
0x0020	arr[8]	90
0x0024	arr[9]	100

Increasing Address

Contiguous Memory
(Each int typically occupies 4 bytes)

4) Initialization

```
int arr[5] = { 10, 20, 30, 40, 50 };
```

Address (Hex)	0x0000	0x0004	0x0008	0x000C	0x0010
Array Element	arr[0]	arr[1]	arr[2]	arr[3]	arr[4]
Value	10	20	30	40	50

Declaration syntax

```
type arrayName[size];
```

Array: Declaration, Index, and Rules

1 SYNTAX – Array Declaration

```
int scores[10];
```

Data type

Type of elements stored in the array

Array name

Name used to access the array

Number of elements

Size of the array (must be a positive integer constant)

2 INDEX

- Index starts at **0** and goes to **size-1**.

Each box can store an integer of type int.

scores[0]	scores[1]	scores[2]	scores[3]	scores[4]
scores[5]	scores[6]	scores[7]	scores[8]	scores[9]

0th element (index 0)

9th element (index 9)

3 EXAMPLES

```
int score[60]; // Array score  
               // with 60 int  
               // type values
```

```
float cost[12]; // Array cost  
               // with 12 float  
               // type values
```

```
char name[50]; // Array name  
              // with 50 char  
              // type values
```



4 RULES – Array Size Must Be a Constant



WARNING

You should always use constants when representing the size of an array.

Using a variable, negative, 0, or a real number as the size of an array is a **compilation error**.

```
int scores[]; // Error! Array size must be specified  
int scores[size]; // Error! Size cannot be a variable  
int scores[-2]; // Error! Size cannot be negative  
int scores[6.7]; // Error! Size must not be a real number
```

5 BEST PRACTICE – Use #define for Array Size

```
#define SIZE 10  
int scores[SIZE];
```



It becomes easy to change the size of the array.



TIP

Usually, when declaring an array, the size of the array is specified as a symbolic constant created with the **#define**.



KEY POINTS



Elements of an array are of the same type.



Stored in **contiguous** memory locations.



Accessed by index (**0 to size-1**).



Efficient for storing and processing related data.

Array Declaration

```
int score[60];    // Array score with 60 int type values
float cost[12];   // Array cost with 12 float type values
char name[50];    // Array name with 50 char type values

// Caution: you should always use CONSTANTS to represent
// the size of an array. Using a variable as the array size
// results in a compilation error. If the size is negative,
// 0, or a real number, it is also a compilation error.
```

Array declaration: data type, array name, and number of elements.

Array Basics Example

```
#include <stdio.h>

int main( void )
{
    int i ;
    int scores[5];

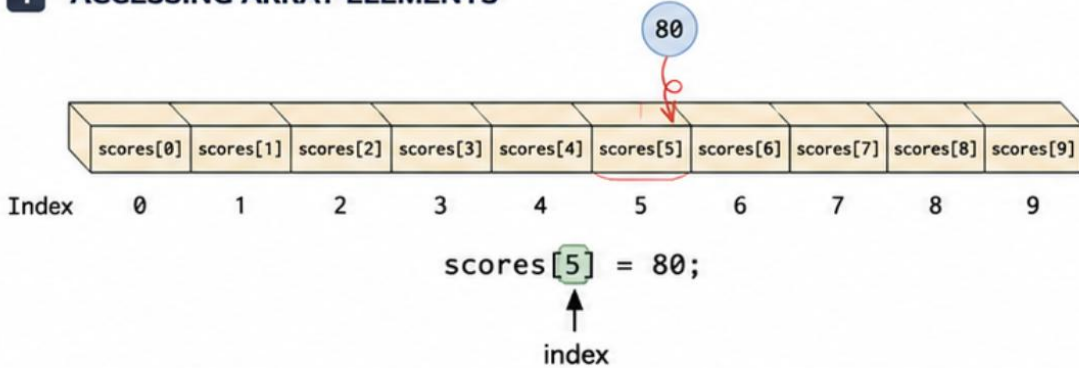
    scores[0] = 10;
    scores[1] = 20;
    scores[2] = 30;
    scores[3] = 40;
    scores[4] = 50;

    for ( i =0; i < 5; i ++){
        printf ( "scores[%d]=%d\n" , i , scores[ i ] );
    }
    return 0;
}
```

Index: the number identifying an array element. The for loop is useful when processing array elements one by one.

Array: Accessing array elements

1 ACCESSING ARRAY ELEMENTS



```
scores[5] = 80;
scores[1] = scores[0];
scores[i] = 100;      // i is an integer variable
scores[i+2] = 100;    // The formula becomes the index.
scores[index[3]] = 100; // index[] is an array of integers
```

2 INITIALIZING AN ARRAY

```
scores[0] = 0;
scores[1] = 0;
scores[2] = 0;
scores[3] = 0;
scores[4] = 0;
```

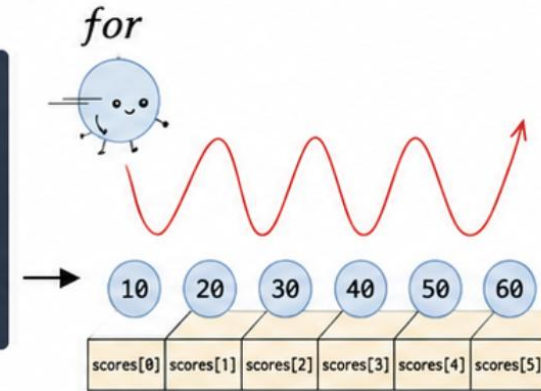
```
#define SIZE 5
...
for (i = 0; i < SIZE; i++)
    scores[i] = 0;
```



Initialize all elements to 0 using a loop!

3 ARRAY EXAMPLE (Increasing Values)

```
scores[0]=10
scores[1]=20
scores[2]=30
scores[3]=40
scores[4]=50
```



4 COMPLETE PROGRAM EXAMPLE

```
#include <stdio.h>
#include <stdlib.h> // srand
#include <time.h>   // time
#define SIZE 5

int main(void)
{
    int i;
    int scores[SIZE];

    srand((unsigned)time(NULL));
    for (i = 0; i < SIZE; i++)
        scores[i] = rand() % 100;

    for (i = 0; i < SIZE; i++)
        printf("scores[%d]=%d\n", i, scores[i]);

    return 0;
}
```



Sample Output (each run may differ)

```
scores[0]=41
scores[1]=67
scores[2]=34
scores[3]=0
scores[4]=69
```

- Index starts at **0** and goes to **size-1**.
- Each element is accessed by its index.
- Arrays store elements of the same type in **contiguous memory** locations.
- Arrays are easy to process using **loops**.

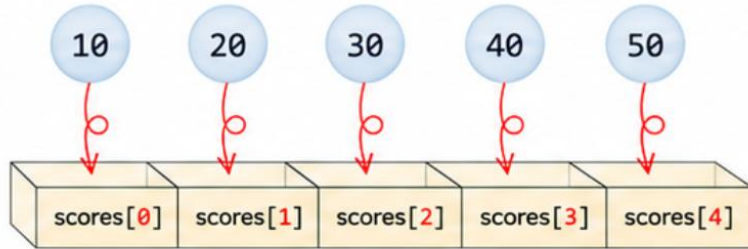
DECLARATION SYNTAX

```
type arrayName[size];
```

Array: Initialization

1 INITIAL VALUES (All values given)

```
int scores[5] = { 10, 20, 30, 40, 50 };
```



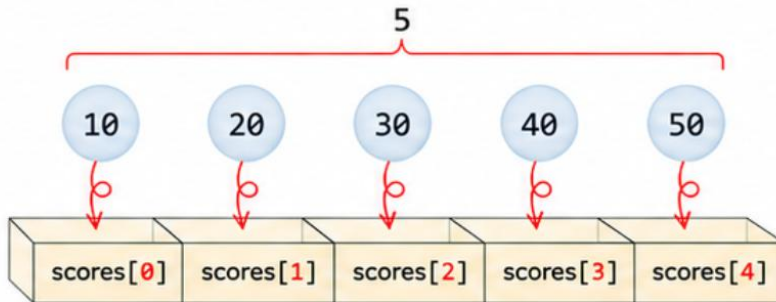
The initial values of the elements are listed within curly brackets, separated by commas.

Access examples

```
scores[5] = 80;           // index 5 → 6th element
scores[1] = scores[0];    // assign value of scores[0] to scores[1]
scores[i] = 100;          // i is an integer variable
scores[i+2] = 100;        // the formula becomes the index
scores[index[3]] = 100;   // index[] is an array of integers
```

3 SIZE FROM INITIAL VALUES

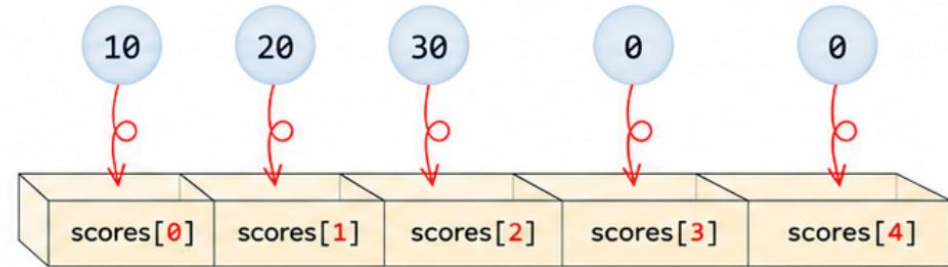
```
int scores[] = { 10, 20, 30, 40, 50 };
```



If the size of the array is not given, the number of initial values becomes the size of the array.

2 PARTIAL INITIALIZATION (Missing values)

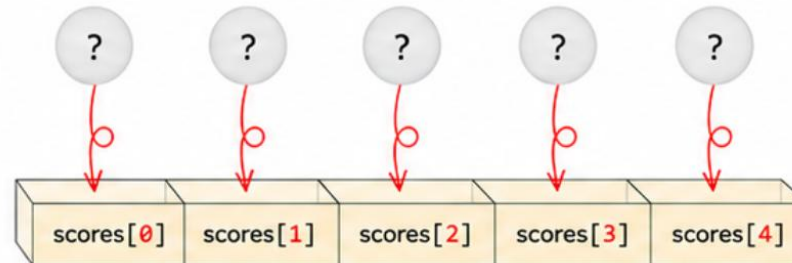
```
int scores[5] = { 10, 20, 30 };
```



If you give only some of the initial values, the remaining elements will be initialized to 0.

4 UNINITIALIZED (No initial values)

```
int scores[5];
```



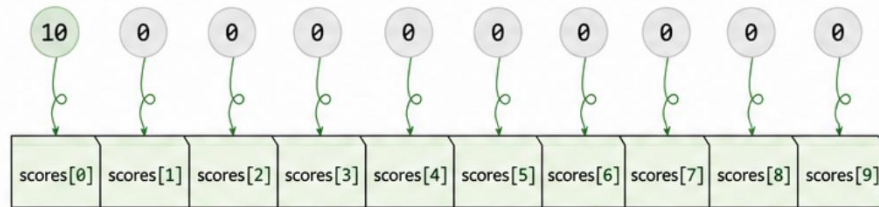
If you do not initialize the array, it contains garbage values (indeterminate values). The values can be anything.

Array: Initialization – Wrong

5 Partial Initialization of an Array

If you try to initialize all elements of an array to 10 as follows, it will be an error.

```
int scores[10] = { 10 };
```



At this time, only the first element becomes 10 and the remaining elements are all 0.

6 Array Initialization Rule

You cannot assign values by enclosing them in braces except when initializing an array. To save, you must assign a value to each array element.

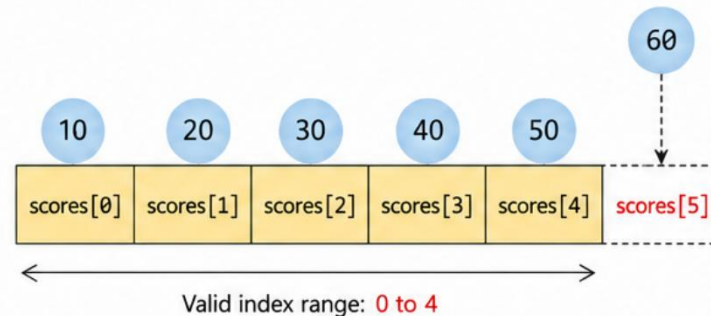
```
#define SIZE 3  
  
int main(void)  
{  
    int scores[SIZE];  
    scores = { 6, 7, 8 }; // Compile error!!  
}
```

This will cause a compile error.

7 Bad Index (Out-of-Bounds) Problem

Array index must be within the valid range: 0 to size-1. Accessing an index outside this range is a **fatal error**.

```
int scores[5];  
...  
scores[5] = 60; // Fatal error!
```



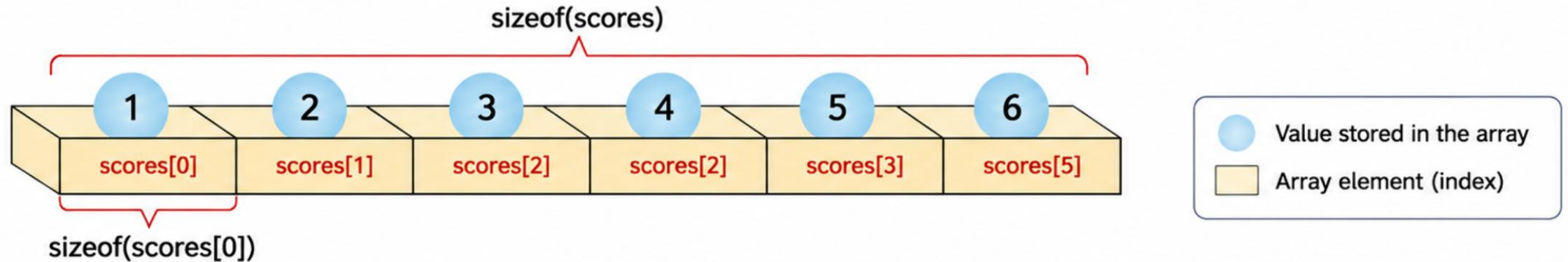
You should not store data where it does not exist.

Initializing an Array

```
int scores[SIZE] = { 31, 63, 62, 87, 14 };  
// If the number of initial values is less than the number of  
// elements, only the elements in front are initialized -  
// all remaining elements are initialized to 0:  
int scores[SIZE] = { 31, 63 };  
  
// If you leave the array size empty, the compiler  
// automatically sets the size to the number of initial values:  
int scores[] = { 31, 63, 62, 87, 14 };  
  
// If no initial value is given (as a local array), elements  
// contain a garbage value - just like an uninitialized variable.
```

Array: Calculating the number of Array Elements

In C, arrays are stored in a contiguous memory block.



```
int scores[] = { 1, 2, 3, 4, 5, 6 };
int i, size;

size = sizeof(scores) / sizeof(scores[0]);

for ( i = 0; i < size; i++ )
    printf ( "%d ", scores[i]);
```

Formula

Number of elements = `sizeof(array) / sizeof(array[0])`

`sizeof(scores)`

Total size (in bytes) of the entire array.
(size of all elements together)

`sizeof(scores[0])`

Size (in bytes) of the first element.
(size of one element)

Example

For `scores[] = { 1, 2, 3, 4, 4, 5, 6 };`
`sizeof(scores) = 6 * sizeof(int)`, `sizeof(scores[0]) = sizeof(int)`
 $\Rightarrow \text{size} = 6 * \text{sizeof(int)} / \text{sizeof(int)} = 6$

OUTPUT

1 2 3 4 4 5 6 (size = 6)

Array: Copying Array Elements

You cannot assign one array to another directly. You must copy each element individually.



Wrong way



```
int a[SIZE] = {1, 2, 3, 4, 5};  
int b[SIZE];
```

```
a = b; // compile error !
```

Why wrong?

In C, an array name represents the address of the first element. Arrays are not single values, so they cannot be assigned.



The right way



```
int a[SIZE] = {1, 2, 3, 4, 5};  
int b[SIZE];  
int i;
```

```
for (i = 0; i < SIZE; i++)  
    a[i] = b[i];
```

Copy each element
one by one



Loop copies elements from a to b using the same index.



Key point: Arrays in C cannot be assigned directly. Always copy elements one by one using a loop.

Array: Comparing Arrays

Arrays cannot be compared using `==`. You must compare each element one by one.



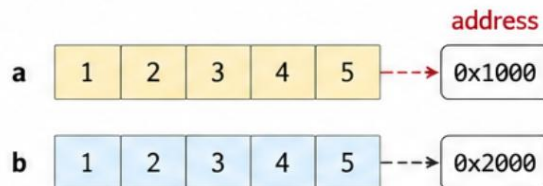
① Incorrect Array Comparison

Comparing array names only compares their addresses, not their contents.

```
#include <stdio.h>
#define SIZE 5

int main(void)
{
    int i;
    int a[SIZE] = { 1, 2, 3, 4, 5 };
    int b[SIZE] = { 1, 2, 3, 4, 5 };
```

```
    if ( a == b ) // Incorrect array comparison
        printf ( "Incorrect result .\n" );
    else
        printf ( "Incorrect result .\n" );
}
```



Result

`a == b` is FALSE
because
`0x1000 != 0x2000`

Output: Incorrect result .



② Correct Array Comparison

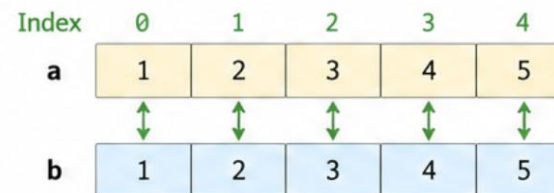
Compare each element one by one.

```
#include <stdio.h>
#define SIZE 5

int main(void)
{
    int i;
    int a[SIZE] = { 1, 2, 3, 4, 5 };
    int b[SIZE] = { 1, 2, 3, 4, 5 };

    for ( i = 0; i < SIZE; i++ ) // Compare each element
    {
        if ( a[i] != b[i] )
        {
            printf ( "a[] and b[] are not equal .\n" );
            return 0;
        }
    }
    printf ( "a[] and b[] are equal .\n" );
    return 0;
}
```

Compare
elements
one by one



All elements are equal
⇒ Arrays are equal

Output: a[] and b[] are equal .

Counting Array Elements, Copying, and Comparing

```
int scores[] = { 1, 2, 3, 4, 5, 6 };
int size = sizeof (scores) / sizeof (scores[0]);    // 6

// Copying an array
int a[SIZE] = {1,2,3,4,5};
int b[SIZE];
// a = b;  // compile error! Arrays cannot be assigned directly.
for (int i = 0; i < SIZE; i++)
    a[i] = b[i];    // copy element by element - the right way

// Comparing arrays: if (a == b) is WRONG (compares addresses)
// Compare element by element instead, in a loop.
```

Lab: Finding the Minimum

- When we buy products on the Internet, we search for the cheapest price through price comparison sites - similar to the problem of finding the minimum value among the integers in an **array**.

- Algorithm

1. Initialize the elements of array `prices[]` to 28 81 60 83 67 10 66 97 37 94.
2. assume that the first element is the minimum value `minimum`.
3. `for( i=1; i < size of array ; i ++)`
4. `if ( prices[ i ] < minimum )`
5. `minimum = prices[ i ]`
6. When the iteration ends, the minimum value is stored in `minimum` .

- Execution result

```
-----  
1  2  3  4  5  6  7  8  9 10  
-----
```

```
28 81 60 83 67 10 66 97 37 94
```

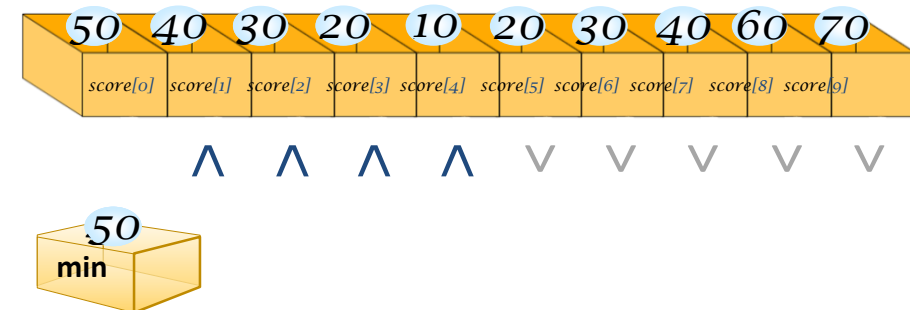
```
The minimum value is 10 .
```

Store	Certified rating	Inventory	Price	Total price
 Your Trusted Source since 1983	★★★★★ Rate this store See store profile	In stock Great Accessory Prices	Price: \$312.00 Tax: \$0.00 Shipping: Free	\$312.00 Your best price Shop now
 OF MAINE Since 1979	★★★★★ Rate this store See store profile	In stock	Price: \$312.95 Tax: \$0.00 Shipping: Free	\$312.95 Shop now
 H&B Photo	★★★★★ Rate this store See store profile	In stock	Price: \$312.95 Tax: \$0.00 Shipping: Free	\$312.95 Shop now
 butterfly ty photo	★★★★★ Rate this store See store profile	In stock	Price: \$313.00 Tax: \$0.00 Shipping: Free	\$313.00 Shop now
 PAV	Not yet rated Rate this store See store profile	In stock	Price: \$316.50 Tax: \$0.00 Shipping: Free	\$316.50 Shop now

Labs: Solution

```
#include <stdio.h>
#define SIZE 10
int main( void )
{
    int prices[SIZE] = {28, 81, 60, 83, 67, 10, 66, 97, 37, 94 };
    int i , minimum;

    minimum = prices[0];
    for ( i = 1; i < SIZE; i ++ ) {
        if ( prices[ i ] < minimum ) {
            minimum = prices[ i ];
        }
    }
    printf ( " The minimum value is %d.\n" , minimum);
    return 0;
}
```



Passing an Array to a Function

The original array is passed, not a copy.
A pointer to its first element is passed.

1 In main()

```
int main(void)
{
    ...
    get_average(  , STUDENTS);
    // Passes the original array (its address)
    ...
}
```

Passes the
address of the
first element

2 In get_average()

```
int get_average(  , int size)
{
    ...
    sum += scores[i]; // Access elements
    ...
}
```

How arrays are passed to functions

When you pass an array to a function,
C automatically converts it to a pointer to the first element.

Array name (arr)	28	81	60	83	67	10	66	97	37	94
(address of arr[0])	0	1	2	3	4	5	6	7	8	9
Index	↑									

So, inside the function, the parameter acts like a pointer (`int *`).

Function prototypes (equivalent)

```
void func(int arr[], int size);
```

is equivalent to

```
void func(int *arr, int size);
```

Key Points

- The array itself is *not copied*.
- A **pointer to the first element** is passed.
- Changes made to the array inside the function affect the original array.



Remember: C passes arguments by value. For arrays, the value passed is the **address of the first element**.

2-Dimensional Array

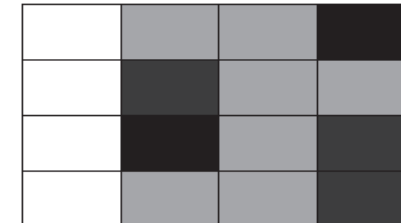
- The data itself is often two-dimensional
 - for example, digital images or board games are fundamentally two-dimensional.

Used when storing data in table or matrix form.

- `int s[10];` // one-dimensional **array**
- `int s[3][10];` // **two-dimensional array** (3 rows, 10 columns)
- `int s[5][3][10];` // three-dimensional **array**

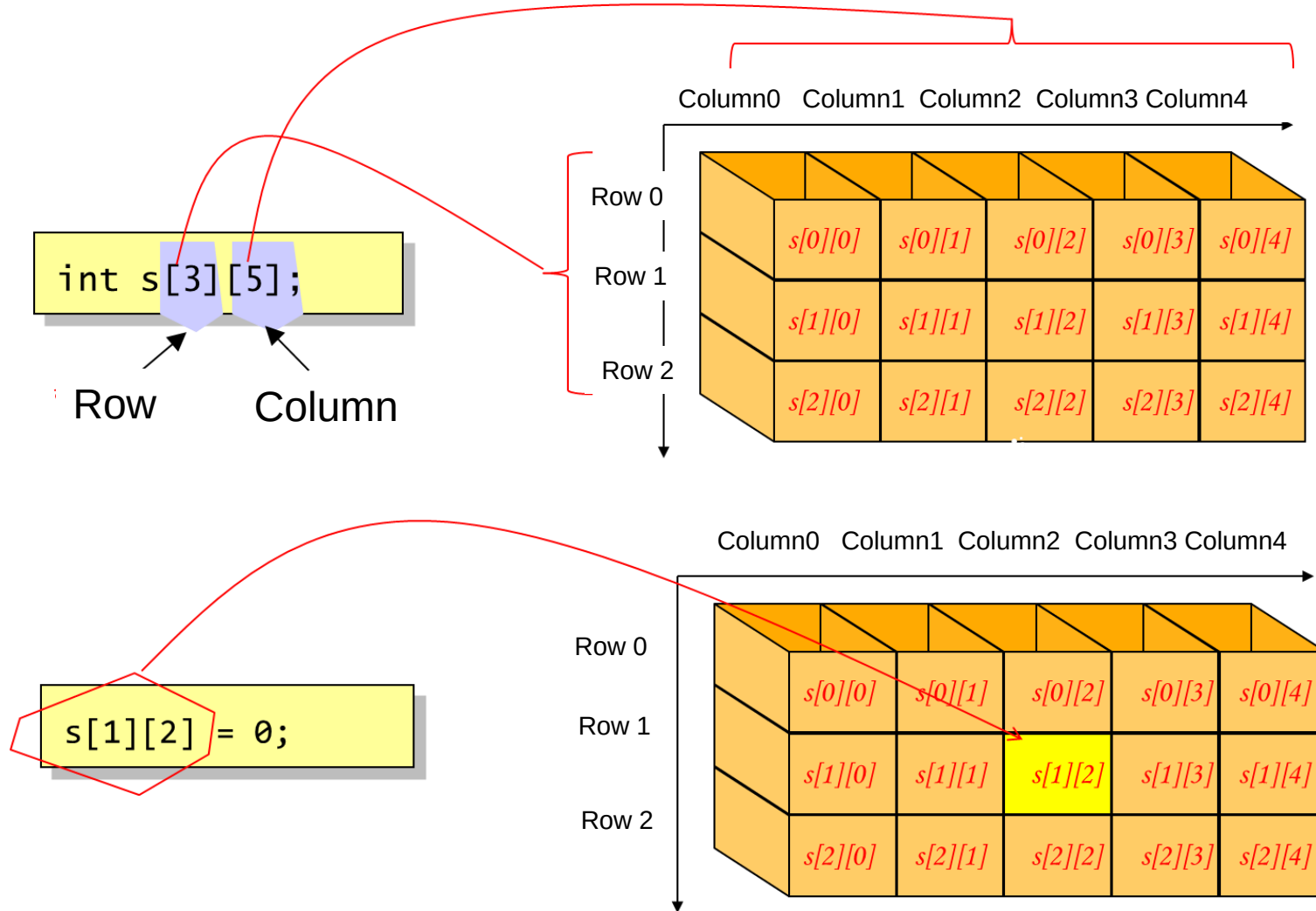
Example: Student Test Scores

Student	Midterm Exam (30%)	Final Exam (40%)	Ranking score (20%)	Quiz Score (10%)	Number of subjects failed/falling
1	87	98	80	75	3
2	95	89	90	91	0
3	65	68	50	49	0



255	120	120	0
255	80	120	120
255	0	120	80
255	120	120	80

2-Dimensional Array: Accessing by Index



2-Dimensional Array: Initialization

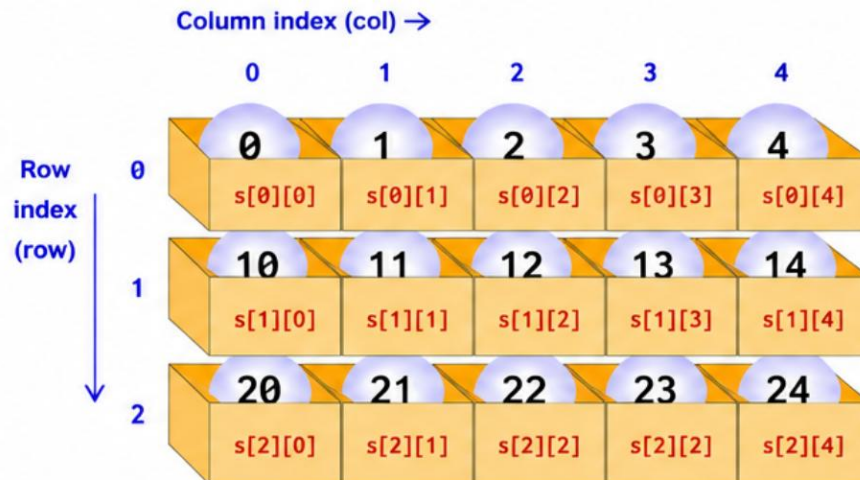
A 2D array in C is stored in **row-major order (row by row)**.

```
int s[3][5] = {  
    { 0, 1, 2, 3, 4 },      // Initial values of elements in the first row  
    { 10, 11, 12, 13, 14 }, // Initial values of elements in the second row  
    { 20, 21, 22, 23, 24 } // Initial values of elements in the third row  
};
```

Array Size

- Rows: 3
- Columns: 5
- Total elements: $3 \times 5 = 15$
- Type: int
- Memory layout: row-major order

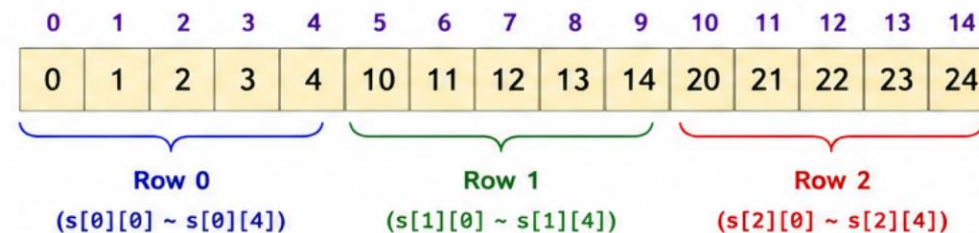
2D Array Layout (s[row][col])



Row-Major Order (Linear View in Memory)

```
int s[][5] = {  
    0, 1, 2, 3, 4, 10, 11, 12, 13, 14, 20, 21, 22, 23, 24  
};
```

Linear index



Key Points

- In C, a 2D array is an array of arrays.
- Elements are stored in **row-major order (row by row)**.
- Access element at row *i*, column *j* using: **s[i][j]**
- Number of rows = 3
- Number of columns = 5
- Total number of elements = 15

Initializing a 2D Array

```
int s[3][5] = {  
    { 0, 1, 2, 3, 4 }, // first row  
    { 10, 11, 12, 13, 14 }, // second row  
    { 20, 21, 22, 23, 24 } // third row  
};
```

// The row size can also be inferred:

```
int s[][5] = {  
    { 0, 1, 2, 3, 4 },  
    { 10, 11, 12, 13, 14 },  
    { 20, 21, 22, 23, 24 }  
};
```

Matrices Using Multidimensional Arrays

```
#include < stdio.h >
#define ROWS 3
#define COLS 3

int main( void )
{
    int A[ROWS][COLS] = { {2,3,0}, {8,9,1}, {7,0,5} };
    int B[ROWS][COLS] = { {1,0,0}, {1,0,0}, {1,0,0} };
    int C[ROWS][COLS];
    int r, c;

    for (r = 0; r < ROWS; r++)
        for (c = 0; c < COLS; c++)
            C[r][c] = A[r][c] + B[r][c];    // matrix addition

    for (r = 0; r < ROWS; r++) {
        for (c = 0; c < COLS; c++) printf ( "%d " , C[r][c]);
        printf ( "\n" );
    }
    return 0;
}
```

Summary: Passing to a Function (1D, 2D, & 3D cases)

1 1D Array

Passing Arrays to Functions in C

- 1D arrays convert to pointers
- `int arr[] == int *arr`
- Array size is not automatically passed

Equivalent function signatures:

- `void func(int arr[]);`
- `void func(int *arr);`
- `void func(int arr[10]);` // size ignored



In C, array parameters are treated as pointers, so the function cannot determine the array size unless it is passed explicitly.

2 2D / 3D Array

For 2D arrays,
the column size must be specified:

- `void func(int arr[][5]);`
- `void func(int (*arr)[5]);`

For 3D arrays,
last two dimensions must be known:

- `void func(int arr[][3][4]);`
- `void func(int (*arr)[3][4]);`

3 Key Points

- 1D: `int arr[]` → pointer
- 2D: `int arr[][C]` → need columns
- 3D: `int arr[][R][C]` → need last two dims

- Use `sizeof` carefully; size is not preserved.

```
void func(int arr[]) {  
    int size = sizeof(arr) / sizeof(arr[0]);  
} // X WRONG!
```



Inside the function, `arr` is a pointer, not the original array.

4 Why Row Count Must Be Passed Separately

- When passing a 2D array to a function: `arr` converts to a pointer to an array of `C` ints. (here `C = 5`)

```
void func(int arr[][5]);  
→ arr is of type int (*)[5]
```

- The function only knows the column size (5). It does **NOT** know the number of rows.
- Therefore, pass row count explicitly:
`void func(int arr[][5], int rows);`



Takeaways

- ✓ Arrays are passed by value in C, but the value passed is the **address of the first element**.
- ✓ 1D arrays degrade to pointers.
- ✓ For 2D arrays, the column size must be specified.
- ✓ For 3D arrays, the last two dimensions must be specified.
- ✓ Use `sizeof` carefully; size information is not preserved.
- ✓ Always pass row count separately for 2D arrays.

Common Mistakes

- C does **not** check array bounds
 - writing past the end silently corrupts nearby memory.
- Arrays are indexed from **0**
 - the last valid index of `a[N]` is **N-1**, not N.
- A fixed-size array's length must be known at **compile time**.
- Multi-dimensional arrays are stored **row-major**
 - row by row - in memory.
- Inside a function that receives an array parameter, **sizeof** no longer gives the array's true size.

Q & A

Lab & Quiz: right after this - see you in the practice session!